
데이터 전처리의 이해

2주차 · 결측치 · 이상치 · 정규화 — 분석의 80%는 전처리다

천재성

반도체장비소프트웨어과 · 전공실습

지난주 복습 (1주차)

- **장비 데이터의 4가지 특징**
 - 실시간·연속성 / 다변량 / 이상이 곧 비용 / 대용량
- **가상 장비 데이터 생성 (온도·압력 센서, 1초 간격)**
 - `pd.DataFrame`, `np.random.normal`, `pd.date_range` 사용
- **오늘의 질문**
 - 지난주 데이터는 '너무 깨끗'했습니다. 실제 장비 데이터에 센서 오류·통신 끊김이 생기면?

2주차 학습 목차

- 1 **전처리란 무엇인가** 왜 80%인가 · Garbage In Garbage Out
- 2 **더러운 데이터 만들기** 실습용 데이터에 결측·이상·중복 주입
- 3 **결측치(Missing) 처리** 탐지 → 삭제 vs 대체 · 실습
- 4 **이상치(Outlier) 처리** IQR·Z-score 탐지 → 제거 · 실습
- 5 **중복·자료형 정리** 중복 제거, 타입 변환 · 실습
- 6 **정규화 / 표준화** 스케일 맞추기 · 실습
- 7 **미니 프로젝트** 더러운 데이터 → 깨끗한 데이터 파이프라인

1

전처리란 무엇인가

Data Preprocessing

데이터 전처리(Preprocessing)란?

수집한 원본(raw) 데이터를 분석·학습에 적합한 형태로 다듬는 모든 과정

- 원본 데이터는 항상 '지저분'하다 — 결측, 이상값, 중복, 단위 불일치
- 전처리 없이 분석하면 → 잘못된 결론 (Garbage In, Garbage Out)
- 비유: 요리 전 '재료 손질'과 같다. 손질 안 된 재료로는 좋은 요리가 안 나온다

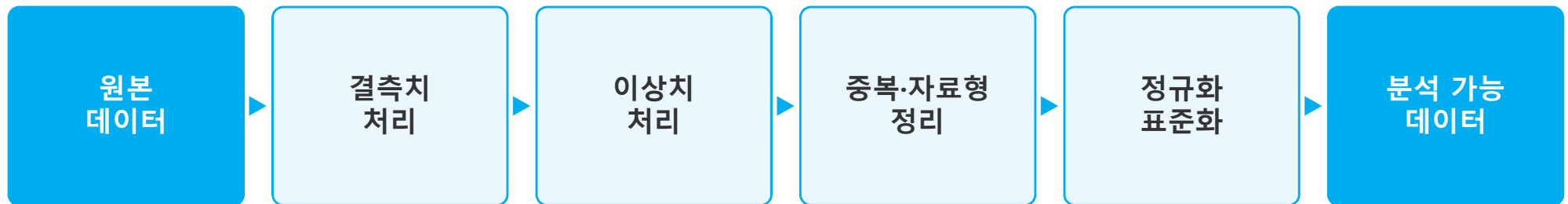
왜 '분석의 80%'라고 할까?

80%

데이터 과학자가 전처리·정제에 쓰는 시간 비중

- 실제 분석/모델링은 20%에 불과
- 데이터가 깨끗할수록 모델 성능이 크게 향상
- 장비 데이터는 센서 오류·통신 끊김이 잦아 전처리 비중이 더 큼
- '좋은 전처리 = 좋은 분석'의 출발점

전처리 전체 흐름



오늘 우리는 이 흐름을 왼쪽에서 오른쪽으로 하나씩 직접 코딩합니다.

1. 전처리란 무엇인가

2

실습용 데이터 만들기

Create a Messy Dataset

왜 데이터를 일부러 '더럽힐까'?

- 아직 실제 장비 데이터가 없다 → 시뮬레이션으로 연습
- 실제 장비에서 흔히 생기는 문제를 의도적으로 재현
 - 센서 통신 끊김 → 결측치 (NaN)
 - 센서 순간 오작동 → 이상치 (말도 안 되는 값)
 - 로그 중복 기록 → 중복 행
 - 수집 시스템 오류 → 숫자가 문자열로 저장
- '문제를 알아야 고칠 수 있다' — 더럽힌 뒤 직접 청소

[실습 2-1] 기본 장비 데이터 생성

```
# preprocessing.py - 1주차 데이터 확장
import pandas as pd
import numpy as np

np.random.seed(42)
n = 200
df = pd.DataFrame({
    'time': pd.date_range('2026-03-01', periods=n, freq='s'),
    'temp': np.random.normal(450, 5, n),      # 온도(°C)
    'pressure': np.random.normal(1.0, 0.05, n), # 압력(Pa)
    'vibration': np.random.normal(0.3, 0.1, n) # 진동
})
print(df.shape)    # (200, 4)
```

→ 온도·압력·진동 3개 센서, 200행짜리 깨끗한 데이터에서 출발합니다.

[실습 2-2] 데이터 더럽히기

```
# 1) 결측치: 30개를 무작위로 NaN 처리
idx = np.random.choice(df.index, 30, replace=False)
df.loc[idx, 'temp'] = np.nan

# 2) 이상치: 5개를 비정상 고온으로
df.loc[df.index[:5], 'temp'] = 9999

# 3) 중복: 앞 3행을 통째로 복제해 추가
df = pd.concat([df, df.iloc[:3]], ignore_index=True)

# 4) 자료형 오류: 압력을 문자열로
df['pressure'] = df['pressure'].astype(str)
df.to_csv('messy.csv', index=False) # 저장
```

→ 이제 'messy.csv'는 실제 현장 데이터처럼 4가지 문제를 모두 가집니다.

[실습 2-3] 데이터 상태 진단

```
# 더러운 데이터 다시 읽기
df = pd.read_csv('messy.csv')

print(df.info())          # 자료형, 결측 개수 한눈에
print(df.isnull().sum()) # 컬럼별 결측치 개수
print(df.duplicated().sum()) # 중복 행 개수
print(df.describe())      # 통계 요약 (이상치 힌트)
```

```
temp          29   ← 결측 29개
duplicated     3   ← 중복 3행
temp max 9999 ← 이상치 의심!
pressure float64 ← CSV 재읽기 후 숫자로 자동 인식됨
```

[실습 2-3] 데이터 상태 진단

```
dtypes: float64(3), object(1)
memory usage: 6.5+ KB
None
time          0
temp          29
pressure       0
vibration      0
dtype: int64
3
```

	temp	pressure	vibration
count	174.000000	203.000000	203.000000
mean	888.917376	1.004723	0.290483
std	2005.700126	0.049144	0.099296
min	436.901274	0.837937	0.052836
25%	446.475030	0.970010	0.220512
50%	450.340228	1.004850	0.291088
75%	453.742832	1.034410	0.356678
max	9999.000000	1.192637	0.607888

3

결측치 처리

Handling Missing Values

결측치(Missing Value)란?

값이 비어 있는 데이터. Pandas에서는 NaN(Not a Number)으로 표시

- 왜 생기나 — 센서 통신 끊김, 측정 실패, 저장 오류
- 왜 문제인가 — 평균·합계 등 계산이 어긋나고, 모델 학습 시 에러
- **두 가지 대응 전략**
 - ① 삭제(Drop): 결측 행/열을 버린다 — 데이터가 충분할 때
 - ② 대체(Imputation): 평균·중앙값·앞뒤값으로 채운다 — 데이터가 아까울 때

[실습 3-1] 결측치 — 삭제 방식

```
# 결측이 있는 행 전체 삭제
df_drop = df.dropna()
print(len(df), '→', len(df_drop))    # 203 → 173

# 특정 컬럼 기준으로만 삭제
df2 = df.dropna(subset=['temp'])

# 결측이 너무 많은 '열' 삭제 (예: 50% 이상)
df3 = df.dropna(axis=1, thresh=len(df)*0.5)
```

주의: 삭제는 간단하지만 데이터가 줄어듭니다. 30개나 사라졌죠?

[실습 3-2] 결측치 — 대체 방식

```
# 평균값으로 채우기
df_mean = df.copy()
df_mean['temp'] = df_mean['temp'].fillna(df_mean['temp'].mean())

# 중앙값으로 채우기
df_median = df.copy()
df_median['temp'] = df_median['temp'].fillna(df_median['temp'].median())

# 직전 값으로 채우기
df_ffill = df.copy()
df_ffill['temp'] = df_ffill['temp'].ffill()
```

→ 장비 데이터는 '직전 값으로 채우기(ffill)'가 자연스러운 경우가 많습니다.

[실습 3-2] 결측치 — 대체 방식

```
print("원본 결측치 개수")
print(df['temp'].isnull().sum())

print("평균 대체 후 결측치 개수")
print(df_mean['temp'].isnull().sum())

print("중앙값 대체 후 결측치 개수")
print(df_median['temp'].isnull().sum())

print("ffill 대체 후 결측치 개수")
print(df_ffill['temp'].isnull().sum())
```

```
dtype: int64
3
```

	temp	pressure	vibration
count	174.000000	203.000000	203.000000
mean	888.917376	1.004723	0.290483
std	2005.700126	0.049144	0.099296
min	436.901274	0.837937	0.052836
25%	446.475030	0.970010	0.220512
50%	450.340228	1.004850	0.291088
75%	453.742832	1.034410	0.356678
max	9999.000000	1.192637	0.607888

```
203 → 174
원본 결측치 개수
29
평균 대체 후 결측치 개수
0
중앙값 대체 후 결측치 개수
0
ffill 대체 후 결측치 개수
0
bfill 대체 후 결측치 개수
0
```

결측치: 삭제 vs 대체, 언제 무엇을?

방법	적합한 경우	장점	단점
삭제 (dropna)	결측이 적고 데이터가 충분할 때	간단·왜곡 없음	데이터 손실, 시계열 끊김
평균/중앙값 대체	수치형, 분포가 안정적일 때	데이터 보존	분산 축소, 이상치에 민감(평균)
ffill / bfill	시계열·연속 데이터(장비)	흐름 자연스러움	급변 구간에서 부정확

장비 데이터(시계열)는 ffill이 가장 자연스러운 경우가 많지만, 정답은 데이터에 따라 다릅니다.

4

이상치 처리

Handling Outliers

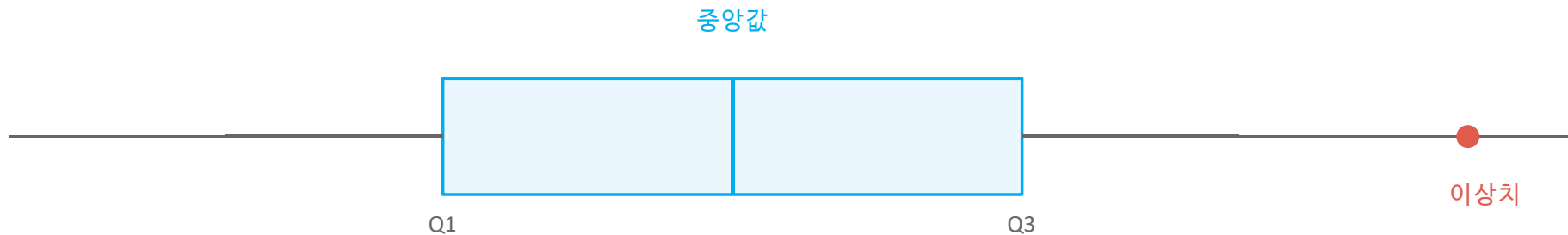
이상치(Outlier)란?

다른 값들과 동떨어진, 비정상적으로 크거나 작은 값

- 장비에서 왜 생기나 — 센서 순간 오작동, 노이즈, 전원 문제
- 왜 중요한가 — 평균·표준편차를 크게 왜곡, 모델 성능 저하
- 단, 모든 이상치가 '오류'는 아니다 — 진짜 고장 신호일 수도 있다
- **대표 탐지법 2가지**
 - ① IQR (사분위 범위) ② Z-score (표준점수)

탐지법 ① IQR (사분위 범위)

- 데이터를 크기순 정렬 후 4등분
- $Q1(25\%), Q3(75\%) \rightarrow IQR = Q3 - Q1$
- 정상 범위: $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$
- 이 범위를 벗어나면 이상치로 판단



[실습 4-1] IQR로 온도 이상치 제거

```
# temp 컬럼에서 결측치는 잠시 제외하고 IQR 계산
temp_data = df['temp'].dropna()
Q1 = temp_data.quantile(0.25)
Q3 = temp_data.quantile(0.75)
IQR = Q3 - Q1
low = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("정상 범위:", low, "~", high)
# 정상 범위 안의 값만 남기기
mask = (df['temp'] >= low) & (df['temp'] <= high)
df_clean = df[mask]
print("원본 행 개수:", len(df))
print("IQR 이상치 제거 후 행 개수:", len(df_clean))
print("이상치 제거 후 temp 최대값:", df_clean['temp'].max())
```

→ → IQR 기준 정상 범위를 벗어난 9999°C 같은 값은 이상치로 걸러집니다.

[실습 4-2] Z-score로 이상치 탐지

- $Z\text{-score} = (\text{값} - \text{평균}) / \text{표준편차} \rightarrow \text{평균에서 몇 } \sigma \text{ 떨어졌나}$

```
# [실습 4-2] Z-score로 이상치 탐지
temp_data = df['temp'].dropna()
mean = temp_data.mean()
std = temp_data.std()
df_z = df.copy()
df_z['z'] = (df_z['temp'] - mean) / std
df_z_clean = df_z[df_z['z'].abs() <= 3]
df_z_clean = df_z_clean.drop(columns='z')
print("Z-score 기준 이상치 제거 후 행 개수:", len(df_z_clean))
print("Z-score 기준 제거 후 temp 최대값:", df_z_clean['temp'].max())
```

IQR vs Z-score: 데이터가 정규분포에 가까우면 Z-score, 치우쳐 있으면 IQR이 유리.

[실습 4-3] 시각화로 눈으로 확인하기

숫자만으로는 감이 안 옵니다. 그래프로 그리면 이상치가 한눈에 보입니다.

```
import matplotlib.pyplot as plt
# [실습 4-3] 시각화로 이상치 확인
import matplotlib.pyplot as plt
# 박스플롯: 이상치가 점으로 튀어나옴
df.boxplot(column='temp')
plt.title('Temperature Boxplot')
plt.show()

# 시계열 선그래프: 9999 지점이 위로 솟구침
df['temp'].plot(title='Temperature over time')
plt.xlabel('Index')
plt.ylabel('Temperature')
plt.show()

# 이상치 제거 후 다시 확인
df_clean['temp'].plot(title='Temperature after IQR Cleaning')
plt.xlabel('Index')
plt.ylabel('Temperature')
plt.show()
```

4. 이상치 처리 → 6주차에서 시각화를 본격적으로 다룹니다. 오늘은 '이상치 확인용'으로 맛보기.

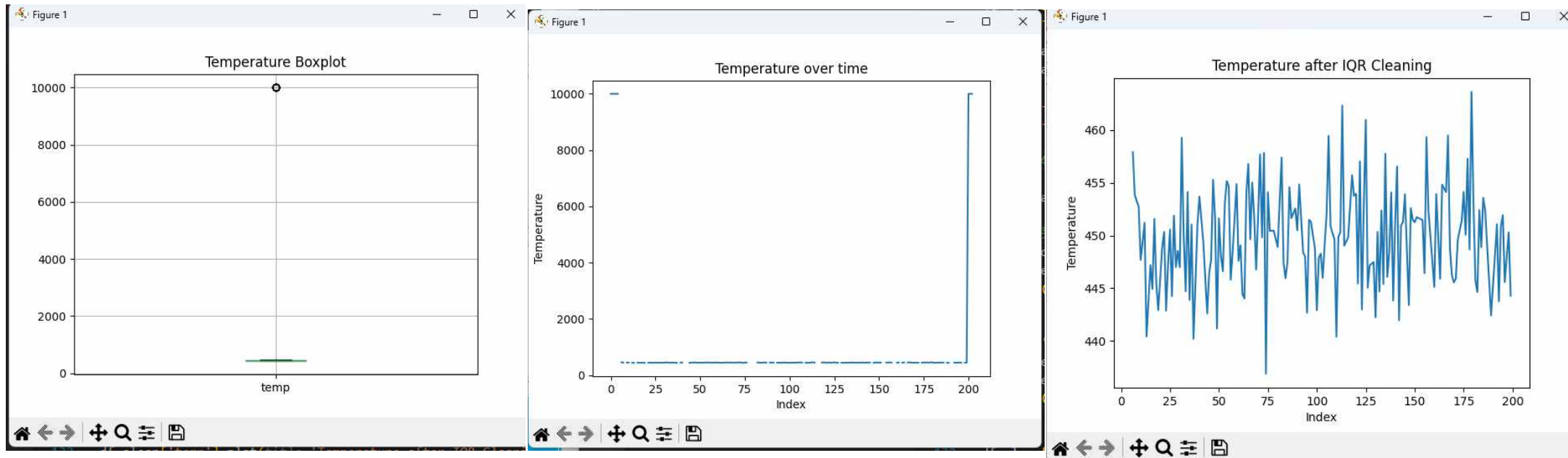
[실습 4-3] 시각화로 눈으로 확인하기

```
260601.py
preprocessing.py
1 # preprocessing
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 np.random.seed(42)
```

원본에서 가져오기 "matplotlib.pyplot"을(를) 확인할 수 없습니다
다. Pylance(reportMissingModuleSource)
(module) matplotlib
문제 보기 (<Alt>+F8) 빠른 수정... (Ctrl+,) ✎ 수정

```
pip install matplotlib
```

[실습 4-3] 시각화로 이상치 제거 전/후 확인하기



5

중복 · 자료형 정리

Duplicates & Data Types

중복 행과 자료형 오류

- **중복(Duplicate) 행**
 - 같은 데이터가 두 번 이상 기록 — 로그 시스템 오류 등
 - 통계를 왜곡(평균이 특정 값으로 쏠림)
- **자료형(Data Type) 오류**
 - 숫자인데 문자열('1.05')로 저장 → 계산 불가
 - 시간인데 문자열로 저장 → 시계열 처리 불가
- 분석 전 반드시 올바른 타입으로 변환해야 한다

[실습 5-1] 중복 제거 & 자료형 변환

```
# 1) 중복 행 제거
print(df.duplicated().sum())    # 3
df = df.drop_duplicates()

# 2) 문자열 → 숫자 변환
df['pressure'] = pd.to_numeric(df['pressure'], errors='coerce')

# 3) 문자열 → 날짜시간 변환
df['time'] = pd.to_datetime(df['time'])
print(df.dtypes)    # 타입 확인
```

errors='coerce' : 변환 불가능한 값은 NaN 처리 → 다시 결측치 처리로 연결!

```
PS D:\program\equip-data-2026> & C:\users\user\AppData\Local\Microsoft\Windows\Shell\6/preprocessing.py
이상치 제거 후 temp 최대값: 463.6008458329481
Z-score 기준 이상치 제거 후 행 개수: 166
Z-score 기준 제거 후 temp 최대값: 463.6008458329481
3
time          datetime64[ns]
temp          float64
pressure      float64
vibration     float64
dtype: object
PS D:\program\equip-data-2026> █
```

전처리 중 자주 만나는 오류

SettingWithCopyWarning

df의 일부를 수정할 때 경고

해결: .loc[]를 명시적으로 사용

TypeError: '>' not supported

문자열과 숫자를 비교

해결: pd.to_numeric으로 먼저 변환

조건 결합 오류

$a > 1 \ \& \ b < 2$ 처럼 괄호 누락

해결: $(a > 1) \ \& \ (b < 2)$ 괄호 필수

FutureWarning: freq='S'

최신 pandas의 단위 표기 변경

해결: freq='s' 소문자 사용

ModuleNotFoundError

라이브러리 미설치

해결: pip install <패키지명>

6

정규화 · 표준화

Normalization & Standardization

왜 스케일을 맞춰야 할까?

- 장비 데이터는 단위·범위가 제각각
 - 온도 450 vs 압력 1.0 vs 진동 0.3 → 숫자 크기 차이가 큼
- 그대로 두면 '큰 숫자'인 온도가 분석·모델을 지배
- 두 가지 방법
 - 정규화(Min-Max): 0~1 사이로 압축
 - 표준화(Standard): 평균 0, 표준편차 1로 변환
- 비유: 키(cm)와 몸무게(kg)를 공정하게 비교하려면 같은 잣대로

[실습 6-1] 정규화 & 표준화

```
# [실습 6-1] 정규화 & 표준화
# 기준 데이터 선택: 이상치 제거 후 데이터 사용
df_scale = df_clean.copy()
# 결측치가 남아 있으면 중앙값으로 대체
df_scale['temp'] = df_scale['temp'].fillna(df_scale['temp'].median())
# 방법 A) Min-Max 정규화
c = df_scale['temp']
df_scale['temp_norm'] = (c - c.min()) / (c.max() - c.min())

# 방법 B) StandardScaler 표준화
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
cols = ['temp', 'pressure', 'vibration']

df_scale[cols] = scaler.fit_transform(df_scale[cols])

print(df_scale[cols].describe())
```

```
pip install scikit-learn
```

[실습 6-1] 정규화 & 표준화

```
Q3: 453.74283209756965
IQR: 7.267802415550989
정상 범위: 435.57332605869215 ~ 464.6445357208961
원본 행 개수: 203
IQR 이상치 제거 후 행 개수: 166
이상치 제거 후 temp 최대값: 463.6008458329481
Z-score 기준 이상치 제거 후 행 개수: 166
Z-score 기준 제거 후 temp 최대값: 463.6008458329481
3
time          datetime64[ns]
temp           float64
pressure       float64
vibration      float64
dtype: object

      temp      pressure      vibration
count  1.660000e+02  1.660000e+02  1.660000e+02
mean   -4.419490e-15  7.651176e-16 -4.226873e-16
std     1.003026e+00  1.003026e+00  1.003026e+00
min    -2.737424e+00 -3.287637e+00 -2.323154e+00
25%    -7.602115e-01 -7.121452e-01 -7.058844e-01
50%     3.544957e-02  9.944392e-03 -3.190641e-02
75%     5.927532e-01  6.192776e-01  6.645556e-01
max     2.895142e+00  3.771066e+00  3.119881e+00
PS D:\program\equip-data-2026> █
```

정규화 vs 표준화 — 무엇을 언제?

정규화 (Min-Max)

- 모든 값을 0~1 범위로 압축
- $(x - \min) / (\max - \min)$
- 최소·최대가 분명할 때 유리
- 이상치에 민감 (max가 9999면 왜곡)
- 예: 이미지 픽셀(0~255 → 0~1)

표준화 (Standard)

- 평균 0, 표준편차 1로 변환
- $(x - \text{평균}) / \text{표준편차}$
- 범위 제한 없음, 분포 모양 유지
- 이상치에 상대적으로 덜 민감
- 예: 대부분의 ML 모델 입력

7

미니 프로젝트

Build a Cleaning Pipeline

미니 프로젝트 — 전처리 파이프라인

- **목표**
 - messy.csv를 입력하면 → 깨끗한 clean.csv를 출력하는 함수 작성
- **포함할 단계 (오늘 배운 전부)**
 - ① 자료형 변환 → ② 중복 제거 → ③ 이상치 제거 → ④ 결측치 대체 → ⑤ 표준화
- **왜 이 순서인가**
 - 타입을 먼저 맞춰야 계산 가능, 이상치를 먼저 빼야 평균 대체가 정확

[실습 7-1] 파이프라인 함수 완성

```
def clean_pipeline(path):  
    df = pd.read_csv(path)  
    # 1) 자료형  
    df['pressure'] = pd.to_numeric(df['pressure'], errors='coerce')  
    df['time'] = pd.to_datetime(df['time'])  
    # 2) 중복 / 3) 이상치(IQR)  
    df = df.drop_duplicates()  
    Q1, Q3 = df['temp'].quantile([.25, .75])  
    iqr = Q3 - Q1  
    df = df[(df['temp'] >= Q1-1.5*iqr) & (df['temp'] <= Q3+1.5*iqr)]  
    # 4) 결측 대체 / 5) 표준화  
    df['temp'] = df['temp'].fillna(df['temp'].median())  
    df.to_csv('clean.csv', index=False)  
    return df
```

`clean_pipeline('messy.csv')` → 한 줄로 전체 전처리 완료!

[실습 7-2] 실행하고 결과 검증하기

```
df = clean_pipeline('messy.csv')

# 검증: 모든 문제가 사라졌는지 확인
print(df.isnull().sum().sum()) # 0 (결측 없음)
print(df.duplicated().sum())  # 0 (중복 없음)
print(df['temp'].max())        # 9999 사라짐
print(df.dtypes)               # 타입 정상
```

```
결측치 합계 : 0
중복 행    : 0
최고 온도  : 약 463 (정상 범위)
pressure   : float64 ✓
```